The AI Execution Layer: A Minimal Runtime Architecture for AI-Native Systems

Wang Luqiang
ai-lib

March 2026

Rapid growth in AI capabilities has produced a fragmented integration landscape in which applications must connect to heterogeneous providers, tools, services, and devices through incompatible APIs. This paper introduces the AI Execution Layer, a minimal runtime architecture that standardizes capability execution while keeping orchestration logic outside the runtime boundary.

The system is formalized as a three-layer logical model, $S = (A, P, E)$, where A denotes cognition, P denotes policy orchestration, and E denotes execution. This logical model is mapped to the broader five-layer AI-Native implementation stack to separate architectural invariants from deployment structure.

Capabilities are unified as $C = (I, O, \Sigma, Q, \Pi)$, where I and O are schemas, Σ specifies execution semantics, Q specifies quality telemetry, and Π specifies pricing metadata. The execution runtime collects and transports Q and pricing metadata without making policy decisions.

Minimality is enforced as a design constraint: the runtime performs deterministic execution functions only, including protocol translation, capability invocation, result normalization, connection management, bounded retry semantics for transient failures, error propagation, and session context forwarding. By excluding planning, workflow orchestration, and provider selection, the runtime remains lightweight, stateless, and horizontally scalable while supporting streaming, polling-based asynchronous tasks, input validation, and baseline security controls.

1 Introduction

1.1 The Integration Challenge

AI applications increasingly depend on heterogeneous capabilities provided by model vendors, external services, and local devices. Providers such as OpenAI [6], Anthropic [1], Google, and DeepSeek expose powerful models through APIs, but direct integration introduces tight coupling and escalating system complexity.

Most current AI frameworks address this challenge through application-level orchestration (e.g., LangChain [5]). Although this approach improves development velocity, it frequently combines orchestration policy and execution mechanics in one runtime, thereby increasing lock-in and reducing component reusability.

1.2 The Missing Layer

The ecosystem still lacks a minimal execution layer dedicated to capability invocation. Such a layer should:

[Figure: Layered architecture diagram - requires tikz package]

Figure 1: Layered architecture of the AI Execution Layer within AI-Native systems, illustrating the separation between cognition, policy orchestration, and capability execution.

1. Execute capabilities defined by standardized protocols.
2. Translate protocol requests into provider-specific API calls.
3. Normalize results into standardized responses.
4. Handle execution errors without implementing application-level retry strategies.

This layer excludes higher-level concerns such as planning, workflow orchestration, and application logic, thereby preserving a lightweight and reusable execution boundary.

1.3 Contributions

The primary contributions are as follows:

1. **Formal Architectural Model:** A three-layer logical model $S = (A, P, E)$ is specified to separate cognition, orchestration, and execution with explicit layer responsibilities.
2. **Minimal Runtime Constraint:** Minimality is formalized as a verifiable design constraint, with explicit runtime responsibilities and non-responsibilities for deterministic capability execution.
3. **Unified Capability Semantics:** Capability is formalized as $C = (I, O, \Sigma, Q, \Pi)$, enabling technical invocation and economic observability to share a single schema-level object.
4. **Cross-Layer Integration:** The three-layer logical model is mapped to the five-layer AI-Native implementation stack, clarifying how execution-layer invariants are preserved in full-system deployments.

The execution layer is then positioned within the broader AI-Native architecture developed in later sections. Figure 1 summarizes the layered boundary, while later figures refine the implementation stack and execution flows.

2 Formal Architectural Model and Problem Formulation

2.1 Logical Model: $S = (A, P, E)$

The AI execution architecture is defined as the logical tuple

$$S = (A, P, E) \tag{1}$$

where:

- A represents the **Agent Intelligence Layer** (reasoning and planning),
- P represents the **Policy Orchestration Layer** (routing and decision-making), and
- E represents the **Execution Layer** (capability invocation).

The model separates the AI system into three distinct layers, each with clearly defined responsibilities.

Agent Intelligence Layer. Agents represent intelligent behavior. They reason about tasks, plan actions, and decide which capabilities should be invoked. This layer handles:

- understanding user goals,
- planning sequences of actions,
- making decisions about capability requirements, and
- managing conversation context and memory.

Policy Orchestration Layer. The policy layer determines how capabilities are selected and composed. This layer is responsible for:

- capability selection based on agent intent,
- provider preference and selection,
- routing strategies,
- cost or latency optimization, and
- policy enforcement.

Execution Layer. The execution layer is responsible only for executing capability calls. It does not implement routing policies, provider selection, or agent logic. This design keeps the runtime minimal and reusable.

2.2 Logical Model vs. Implementation Stack

In this paper, Figure 1 is the **logical model**. The five-layer AI-Native stack presented later is an **implementation stack** used to realize this model in production systems.

- The logical model specifies invariants and boundaries.
- The implementation stack specifies deployment surfaces and engineering composition.

This distinction avoids a common modeling conflict: a system can be five-layer in implementation while remaining three-layer in execution logic.

2.3 Rationale for Separation

This separation addresses a critical architectural problem observed in existing frameworks. Many systems conflate orchestration responsibilities with execution mechanisms, resulting in:

- **Framework lock-in:** applications become tightly coupled to specific orchestration frameworks.
- **Limited reusability:** runtime components cannot be easily reused across different applications.
- **Complexity creep:** execution runtimes grow to include policy logic, making them difficult to maintain.

By separating these concerns, we enable:

- **Runtime portability:** execution runtimes can be reused across different orchestration strategies.
- **Policy flexibility:** organizations can implement custom routing policies without modifying the runtime.
- **Clear boundaries:** each component has well-defined inputs, outputs, and responsibilities.

3 Why Minimal Execution Matters

Minimality is treated as a system constraint rather than a stylistic preference. Embedding policy logic into the execution runtime introduces state coupling, reduces portability, and increases cross-layer failure propagation.

Definition 1 (Minimality Constraint)**.** *The execution layer* ***must*** *implement deterministic capability invocation only, and* ***must not*** *perform policy-driven decisions such as provider selection, strategic retry, or workflow-level fallback.*

Under this constraint, runtime semantics are specified through core responsibilities and explicit non-responsibilities.

3.1 Core Responsibilities

The runtime serves as a standardized capability execution engine. Its responsibilities are intentionally limited to six core functions:

1. **Capability Invocation:** execute a capability defined in the AI Protocol. The runtime receives standardized capability requests and invokes the appropriate execution path.
2. **Protocol Translation:** convert protocol-level requests into provider-specific API calls. This translation layer enables applications to work with standardized capability definitions while the runtime handles provider-specific implementations.
3. **Result Normalization:** return standardized responses to the caller. Regardless of which provider executed the capability, the runtime normalizes results into a consistent format.
4. **Connection Handling:** manage network communication with providers, including connection pooling, timeout management, and network-level error handling.
5. **Error Propagation:** report execution errors without implementing application-level retry strategies. The runtime distinguishes between execution errors, which it reports, and retry logic, which belongs in the orchestration layer.
6. **Session Transport:** forward session context without interpreting application policy. The runtime transports contextual information required for capability execution but does not interpret or modify policy decisions.

Table 1: Execution-runtime responsibilities and explicit exclusions.

Category	Included in the execution runtime	Explicitly excluded from the execution runtime
Invocation path	Capability invocation, protocol translation, connection handling, and result normalization	Capability planning and multi-step workflow design
Failure handling	Error propagation and bounded network-level micro-retries	Provider fallback and orchestration-level retry policy
Context handling	Session-context transport and execution metadata emission	Business-policy interpretation and application-specific logic
Selection logic	Deterministic execution of the supplied request	Provider selection, routing optimization, and policy evaluation

Optional bounded micro-retry semantics. For transient network failures such as connection timeouts or HTTP 429 rate limits, the runtime may implement limited micro-retries—typically one or two immediate retries with exponential backoff at the network layer. This differs from application-level retry strategies in that micro-retries are bounded and automatic, address only transient network failures, do not implement policy-driven retry logic such as provider fallback, and must remain within the overall execution timeout budget.

This choice reflects production constraints: transient network failures are common, and surfacing every transient event to the orchestration layer introduces avoidable latency and cross-layer communication overhead.

3.2 What the Runtime Does Not Do

To maintain its minimal design, the runtime explicitly avoids:

- **Agent Planning:** deciding what sequence of capabilities to invoke.
- **Workflow Orchestration:** managing complex multi-capability workflows.
- **Application Logic:** implementing application-specific business logic.
- **Provider Selection:** choosing which provider should execute a capability.
- **Routing Policies:** implementing cost optimization or latency-based routing.
- **Application-Level Retry Strategies:** determining when and how to retry failed executions across providers or workflows.

These responsibilities belong in the orchestration and agent layers, maintaining clear separation of concerns.

4 Theoretical Foundation: Four Core Principles

4.1 Principle 1: Capability Abstraction

Principle. *An AI agent should depend on capabilities rather than concrete providers or APIs.*

Table 2: Comparison between capability and related abstractions.

Concept	Essence	Role
Function	Computation	Algorithmic operation
API	Remote interface	Access mechanism
Tool	Agent-callable function	Invocation interface
Capability	**Executable ability**	**Real-world action**

Formal definition. A capability is defined as

$$C = (I, O, \Sigma, Q, \Pi) \tag{2}$$

where:

- I is the input schema,
- O is the output schema,
- Σ is the execution semantics,
- Q is the quality telemetry schema, and
- Π is the pricing and billing schema.

A provider implements a capability, and agents invoke capabilities through

$$\text{Invoke}(C, \text{Input}) \rightarrow \text{Output}. \tag{3}$$

Rather than invoking provider-specific APIs directly, agents express the executable ability they require. This abstraction enables provider substitution without agent modification, multi-provider capability implementations, and standardized capability discovery and invocation.

Capabilities represent abilities, whereas APIs represent access mechanisms. This distinction is fundamental to the AI-Native architecture. In this paper's scope, the execution runtime does not optimize Q or Π; it only measures, normalizes, and forwards these fields for policy decisions in the orchestration layer.

4.2 Principle 2: Cognition–Action Separation

Principle. *AI cognition and capability execution should be separated by an action layer.*

Architecture. The architecture can be summarized as:

Cognition Layer (reasoning, planning, memory)
↓
Action Layer (routing, execution)
↓
Capability Providers

Rationale. Without this separation, agents must directly call APIs, tools, and services, leading to tight coupling to specific providers, low portability across execution environments, and ecosystem fragmentation. The action layer mediates between cognitive processes that decide what to do and execution processes that perform the action, enabling agents to focus on reasoning while the execution layer handles capability-invocation mechanics.

[node distance=1.2cm and 0.8cm, capnode/.style=draw, rounded corners=2pt, minimum width=2.0cm, minimum height=0.8cm, align=center, fill=blue!6, exec/.style=draw, rounded corners=2pt, minimum width=1.2cm, minimum height=0.6cm, align=center, fill=green!8, >=Latex] [capnode] (search) `search`; [capnode, right=of search] (sum) `summarize`; [capnode, right=of sum] (gen) `generate_text`; [capnode, right=of gen] (trans) `translate`; [capnode, right=of trans] (pub) `publish`;
[exec, below=of search] (e1) exec; [exec, below=of sum] (e2) exec; [exec, below=of gen] (e3) exec; [exec, below=of trans] (e4) exec; [exec, below=of pub] (e5) exec;
[->] (search) – (sum); [->] (sum) – (gen); [->] (gen) – (trans); [->] (trans) – (pub);
[->] (search) – (e1); [->] (sum) – (e2); [->] (gen) – (e3); [->] (trans) – (e4); [->] (pub) – (e5);

Figure 2: Capability graph for a multi-step publishing task.

4.3 Principle 3: Capability Routing

Principle. *Capability execution should be dynamically routed based on context.*

Routing function.

$$\text{Route}(C, \text{Context}) \rightarrow \text{Provider} \tag{4}$$

Context parameters.

- **Latency:** response-time requirements.
- **Cost:** budget constraints.
- **Policy:** organizational or regulatory policies.
- **Availability:** provider uptime and reliability.
- **Security:** data handling and privacy requirements.

For example, for a capability such as `translate`, available providers may expose different cost and latency trade-offs. The routing system dynamically selects the provider based on context. This mechanism is analogous to DNS routing or service-mesh routing [3], but the routed object here is a capability rather than a network request.

4.4 Principle 4: Capability Graph Execution

Principle. *Complex AI tasks can be modeled as execution over a capability graph.*

Formal definition. A capability graph is defined as

$$G = (\text{Capabilities}, \text{Dependencies}) \tag{5}$$

and is constrained to a directed acyclic graph for execution safety and deterministic ordering.

Example. The task "Write and publish a blog post" can be represented as the capability graph shown in Figure 2.

Execution process.

1. The agent reasons about the task and constructs the capability graph.
2. The orchestration layer determines execution order and provider selection.
3. The execution runtime invokes each capability in sequence.
4. Results flow through the graph, with each capability receiving inputs from predecessor nodes.

This model enables complex tasks to be decomposed into composable capability sequences, while the execution runtime remains responsible only for the mechanics of invocation.

5 AI Protocol and Capability Definition

5.1 Protocol Role

The AI Protocol defines how capabilities are described. It specifies:

- **Providers:** entities offering AI or non-AI capabilities.
- **Models:** specific AI models with defined capabilities.
- **Capability Types:** categories of capabilities such as chat, embedding, and image generation.
- **Endpoints:** where capabilities can be invoked.
- **Parameters:** input and output schemas.

The protocol serves as the contract boundary between orchestration and execution: orchestration decides policy, while execution enforces invocation semantics.

The protocol is typically represented as structured manifests:

Listing 1: Provider-centric protocol manifest organization.

```
provider/
|- metadata.yaml
\- models/
   |- model_a.yaml
   \- model_b.yaml
      \- capabilities/
         |- chat.yaml
         \- embedding.yaml
```

5.2 Capability Definition

Each capability is defined with:

- **Name:** a semantic identifier such as `generate_text` or `search_web`.
- **Input Schema:** a structured input specification.
- **Output Schema:** a structured output specification.
- **Execution Semantics:** the invocation and runtime behavior contract.

- **Quality Metadata (Q):** measured quality fields emitted by execution.
- **Pricing Metadata (Π):** metering and billing fields emitted by execution.
- **Provider Metadata:** information about the provider offering this capability.

Listing 2: Example capability definition.

```
name: generate_text
description: Generate text based on a prompt
input_schema:
  type: object
  properties:
    prompt:
      type: string
      description: The input prompt
    max_tokens:
      type: integer
      description: Maximum tokens to generate
  required: [prompt]
output_schema:
  type: object
  properties:
    text:
      type: string
      description: Generated text
execution:
  endpoint: /v1/completions
  method: POST
quality:
  telemetry_fields: [execution_latency_ms, retry_count]
pricing:
  billing_model: per_token
  currency: USD
provider:
  name: Example Provider
  version: 1.0.0
```

5.3 Provider-Centric Organization

An important design decision is that capability descriptions remain provider-centric rather than capability-centric. The manifest structure follows:

$$\texttt{Provider} \rightarrow \texttt{Models} \rightarrow \texttt{Capabilities}$$

rather than

$$\texttt{Capability} \rightarrow \texttt{Providers}.$$

The runtime can dynamically construct a capability index:

Listing 3: Runtime-generated capability index view.

```
chat/
|- provider A
```

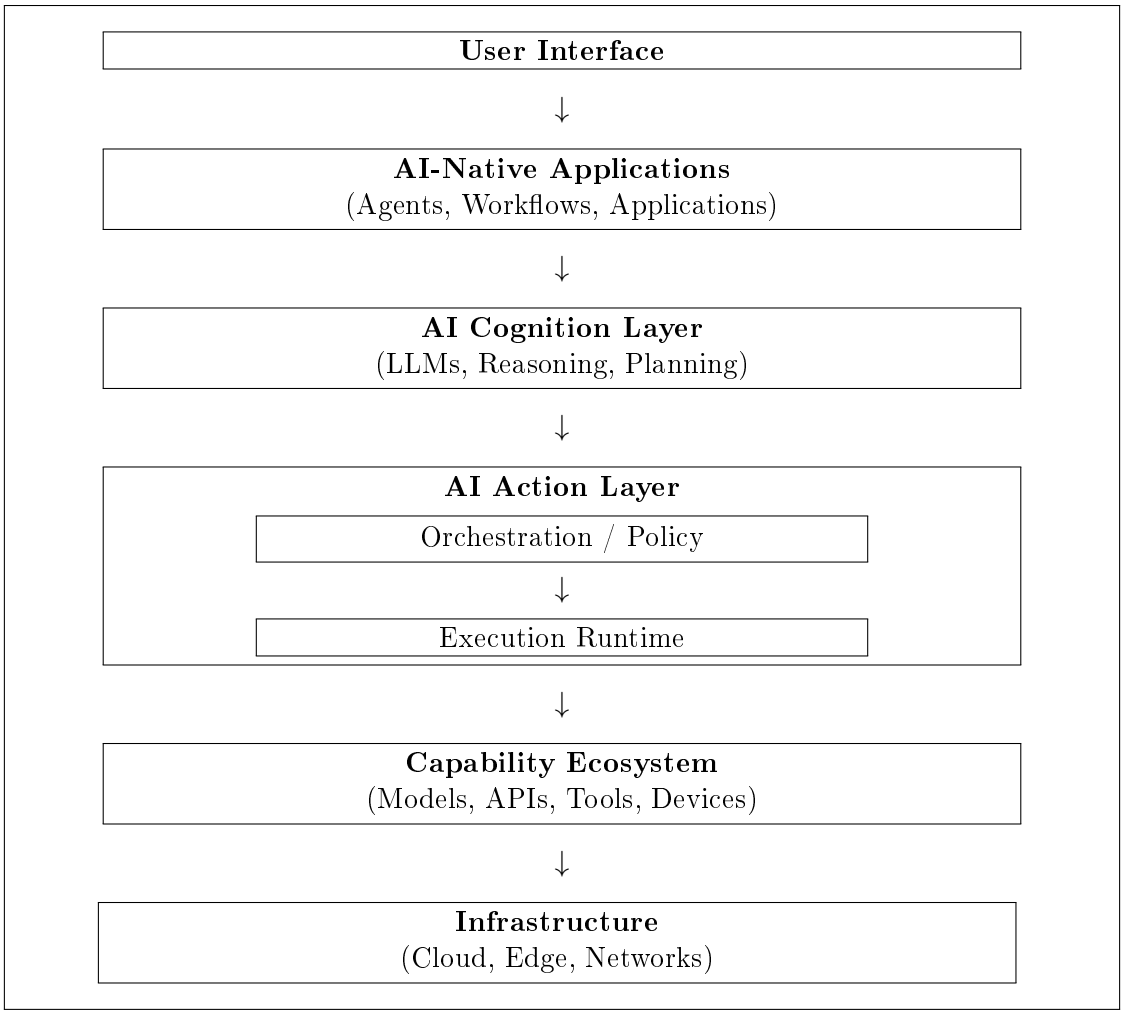

Figure 3: Five-layer AI-Native implementation stack with the action layer decomposed into orchestration and execution.

```
|- provider B
\- provider C
embedding/
|- provider A
\- provider D
```

This index can be rebuilt periodically and optimized based on user preferences, latency measurements, cost calculations, and reliability metrics.

6 Execution Layer Position in the AI-Native Stack

6.1 Five-Layer Architecture

The execution layer sits within a broader AI-Native implementation stack. In the final arXiv bundle, the stack overview will be supplied as a dedicated vector figure. The current conversion pass embeds a LaTeX-native schematic to preserve semantics and compile without external graphics.

Table 3: Mapping between the logical model and the implementation stack.

Logical element	Implementation mapping	Responsibility
A	AI Cognition Layer	Reasoning, planning, memory, and task decomposition.
P	Policy Orchestration Layer	Routing, provider selection, policy evaluation, and workflow control.
E	Execution Runtime	Deterministic capability invocation, protocol translation, result normalization, and execution telemetry forwarding.

6.2 Action Layer Decomposition

A critical insight is that the action layer comprises two subcomponents:

Orchestration component. The orchestration component is responsible for capability routing, provider selection, policy evaluation, and environment selection.

Execution component. The execution component is responsible for API invocation, tool execution, error handling, and result normalization.

This decomposition maintains clear separation of concerns. The orchestration layer makes decisions, such as which provider or environment to use, while the execution layer performs actions, such as invoking APIs and returning normalized results.

When mapped to the logical model in Figure 1, the orchestration component corresponds to P, and the execution runtime corresponds to E.

6.3 Defining Statement

> *The execution layer is the action interface between AI cognition and the real-world capability ecosystem.*

This definition captures the essential role of the layer: mediating between what agents decide to do and how those decisions are realized through capability execution.

7 Architectural Benefits

7.1 Decoupling

Applications become independent of specific providers. Agents express capability requirements without needing to know which provider will fulfill them. This decoupling enables:

- provider substitution without application changes,
- multi-vendor strategies, and
- graceful degradation when providers are unavailable.

7.2 Portability

AI applications can run across different runtime environments. As long as runtimes implement the AI Protocol, applications can be deployed without modification. This portability is analogous to how container images can run on any compatible container runtime [4, 2].

7.3 Capability Discovery

New providers can register capabilities without modifying applications. The capability registry serves as a dynamic discovery mechanism, enabling:

- new capability providers to enter the ecosystem,
- applications to discover new capabilities at runtime, and
- ecosystem growth without centralized coordination.

7.4 Ecosystem Growth

Standard protocols enable third-party extensions and tools. The minimal runtime design creates clear extension points:

- custom capability providers,
- alternative orchestration strategies,
- specialized runtime implementations, and
- monitoring and observability tools.

8 Related Work

8.1 AI Orchestration Frameworks

Frameworks such as LangChain [5] provide practical orchestration abstractions, but often couple orchestration policy and execution mechanics in one framework runtime. This coupling improves developer convenience while weakening boundary clarity and runtime portability.

8.2 Workflow Engines

Workflow engines such as Airflow and Temporal provide robust DAG execution and fault handling. However, their abstraction unit is typically task- or workflow-level orchestration, rather than capability-level semantic invocation with provider-agnostic protocol contracts.

8.3 Service Mesh and RPC Systems

Service meshes and RPC infrastructures provide routing, retries, and observability at network-request granularity. By contrast, the AI Execution Layer operates at capability semantics and explicitly separates policy routing from deterministic invocation.

8.4 Serverless and Function Runtimes

Function-as-a-service platforms provide scalable execution environments, but do not standardize capability abstraction, capability manifests, or cross-provider semantic equivalence.

8.5 Position of This Work

This paper contributes a minimal execution boundary for AI-Native systems:

- protocol-first capability invocation,
- strict cognition–policy–execution separation,
- runtime-level portability across providers and languages, and
- explicit non-responsibilities that prevent policy leakage into execution.

8.6 Multi-Language Implementation

A key validation of the minimal runtime design is its implementability across multiple programming languages. Reference implementations exist in Rust, Python, TypeScript, and Go, demonstrating that the abstraction is not tied to a single language ecosystem. Cross-language consistency is maintained through shared protocol specifications, language-agnostic capability schemas, standardized request/response formats, and common behavioral test suites.

This multi-language support strengthens the claim that the minimal runtime abstraction is ecosystem-oriented rather than framework-specific.

9 Implementation Considerations

9.1 Runtime Interface

The execution runtime exposes a minimal interface:

Listing 4: Minimal execution runtime interface.

```
interface ExecutionRuntime {
    Result execute(CapabilityRequest request)
    List<Capability> listCapabilities()
    HealthStatus healthCheck()
}
```

9.2 Capability Request Format

Capability requests follow a standardized format:

Listing 5: Standardized capability request format.

```
{
  "capability": "generate_text",
  "version": "1.0",
  "input": {
    "prompt": "Explain quantum computing",
    "max_tokens": 500
  },
  "context": {
    "session_id": "abc123",
    "request_id": "req456"
  },
  "provider": {
```

```
    "name": "provider_a",
    "endpoint": "https://api.provider-a.com"
  }
}
```

9.3 Result Format

Results are normalized to a standard structure with execution metrics:

Listing 6: Normalized execution result with telemetry.

```
{
  "capability": "generate_text",
  "status": "success",
  "output": {
    "text": "Quantum computing is...",
    "tokens_used": 450
  },
  "metadata": {
    "provider": "provider_a",
    "latency_ms": 1250,
    "translation_latency_ms": 15,
    "retry_count": 0,
    "timestamp": "2026-03-15T10:30:00Z"
  }
}
```

The `retry_count` and `translation_latency_ms` fields provide visibility into execution behavior, enabling the Policy Orchestration Layer to make data-driven routing decisions.

10 Future Directions

10.1 Capability Discovery Protocol

Future work could define a standardized capability discovery protocol enabling dynamic capability registration, capability health monitoring, and automatic capability indexing.

10.2 Application Packaging Standard

A standardized application packaging format would enable portable AI application distribution, standard deployment models, and explicit capability requirement declaration.

10.3 Capability Marketplace

As capabilities become standardized, marketplaces may emerge for premium capability offerings, capability quality ratings, and usage-based pricing.

10.4 Edge Runtime Support

The execution-layer architecture naturally supports edge-computing scenarios. Runtimes can operate locally on personal devices, home servers, and embedded systems while connecting to local models, local automation systems, and cloud capabilities. This architecture enables personal AI infrastructure, where users operate local runtimes orchestrating personal capabilities.

10.5 Asynchronous Notification Support

While the current design relies on polling for long-running tasks, future runtime versions may support callback-based notifications. In this model:

- providers register webhook endpoints in their manifests,
- the runtime translates provider-specific callbacks into protocol-standard events, and
- the Policy Orchestration Layer subscribes to capability-completion events rather than polling.

This extension would reduce connection overhead for high-concurrency scenarios and enable true event-driven capability orchestration.

11 Limitations and Evaluation Agenda

This work focuses on architectural formalization and runtime boundaries. It does not provide a full-scale benchmark study of latency overhead, translation-cost distribution, or end-to-end economic efficiency under production traffic.

A complete systems evaluation should include:

1. **Translation Overhead:** protocol-to-provider transformation latency and variance.
2. **Runtime Scalability:** throughput and tail latency under mixed synchronous and streaming workloads.
3. **Failure Behavior:** bounded-retry impact and error propagation under provider instability.
4. **Policy–Execution Separation Cost:** control-plane/data-plane interaction overhead under dynamic routing.

These measurements are left to subsequent experimental work and cross-runtime benchmark suites.

12 Conclusion

The emergence of AI agents requires a dedicated architectural boundary for capability execution. This paper introduces the AI Execution Layer as a minimal runtime architecture for agent-driven systems.

The architecture separates cognition, orchestration, and execution through the logical model $S = (A, P, E)$. Within this boundary, the runtime is constrained to six execution responsibilities:

1. capability invocation,
2. protocol translation,
3. result normalization,
4. connection handling,
5. error propagation, and
6. session transport.

Figure 4: Complete execution flow from agent intent to normalized task result.

Figure 5: Execution error propagation and policy handoff.

The theoretical foundation is defined by four principles:

1. **Capability Abstraction:** agents depend on capabilities rather than providers.
2. **Cognition–Action Separation:** intelligence and execution are separated by an action layer.
3. **Capability Routing:** execution is dynamically routed based on context.
4. **Capability Graph Execution:** complex tasks decompose into capability graphs.

The execution layer functions as the action interface between AI cognition and the real-world capability ecosystem. This boundary decouples agents from providers, improves application portability, and supports ecosystem-scale growth.

More broadly, the proposed layer is analogous to operating-system and RPC boundaries in earlier computing eras: it provides a stable execution contract across heterogeneous providers while preserving higher-layer policy autonomy.

As the ecosystem matures, this boundary is expected to become a foundational component of AI-Native software infrastructure.

A Capability Execution Sequence Diagrams

A.1 Complete Execution Flow

The complete execution flow from agent intent to provider response is shown in Figure 4.

A.2 Error Handling Flow

The error propagation path is shown in Figure 5. The diagram preserves the bounded micro-retry distinction between network-level execution behavior and orchestration-level retry policy.

B Capability Definition Examples

B.1 Text Generation Capability

Listing 7: Example text-generation capability manifest.

```
name: generate_text
version: "1.0"
description: Generate text based on a prompt
provider: openai
model: gpt-5.4
input_schema:
  type: object
  properties:
    prompt:
```

```
      type: string
    max_tokens:
      type: integer
      default: 1000
    temperature:
      type: number
      default: 0.7
output_schema:
  type: object
  properties:
    text:
      type: string
    tokens_used:
      type: integer
execution:
  endpoint: /v1/chat/completions
  method: POST
  timeout_ms: 30000
```

B.2 Web Search Capability

Listing 8: Example web-search capability manifest.

```
name: search_web
version: "1.0"
description: Search the web for information
provider: search_service
input_schema:
  type: object
  properties:
    query:
      type: string
    num_results:
      type: integer
      default: 10
output_schema:
  type: object
  properties:
    results:
      type: array
      items:
        type: object
        properties:
          url:
            type: string
          title:
            type: string
          snippet:
            type: string
execution:
  endpoint: /v1/search
  method: GET
```

C Runtime Pseudocode

C.1 Execute Capability

Listing 9: Pseudocode for execution-runtime capability invocation.

```
def execute_capability(request: CapabilityRequest) -> Result:
    """
    Execute a capability request.

    Args:
        request: Standardized capability request

    Returns:
        Normalized execution result
    """
    # 1. Validate request
    validate_request(request)

    # 2. Translate to provider-specific format
    provider_request = translate_request(request)

    # 3. Invoke provider API
    try:
        provider_response = invoke_provider(
            endpoint=request.provider.endpoint,
            data=provider_request
        )
    except ConnectionError as e:
        # Propagate error (no retry logic)
        raise ExecutionError(
            capability=request.capability,
            error_type="connection",
            message=str(e)
        )

    # 4. Normalize response
    result = normalize_response(
        capability=request.capability,
        provider_response=provider_response
    )

    # 5. Return standardized result
    return result
```

C.2 Request Translation

Listing 10: Pseudocode for protocol-to-provider request translation.

```
def translate_request(request: CapabilityRequest) -> dict:
    """
    Translate standardized request to provider-specific format.
```

```
    Args:
        request: Standardized capability request

    Returns:
        Provider-specific request payload
    """
    capability_manifest = load_manifest(request.capability)
    provider_adapter = get_adapter(request.provider.name)

    return provider_adapter.translate(
        capability=request.capability,
        input_data=request.input,
        manifest=capability_manifest
    )
```

References

[1] Anthropic. Claude api reference. https://docs.anthropic.com, 2024.

[2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.

[3] HashiCorp. Consul service mesh. https://www.consul.io/docs, 2024.

[4] Kubernetes. Kubernetes architecture. https://kubernetes.io/docs/concepts/architecture/, 2024.

[5] LangChain. Langchain documentation. https://python.langchain.com/docs, 2024.

[6] OpenAI. Gpt-4 api documentation. https://platform.openai.com/docs, 2023.